

The Connected p -Center Problem on Graphs with Lengths on Edges and Weights on Vertices

William C-K Yen, Shung C-S Yang, and S-C Hu

Department of Information Management

Shih Hsin University, Taipei, Taiwan

Email: {ckyen001@ms7.hinet.net, shung007@hotmail.com, schu@cc.shu.edu.tw}

Abstract

Suppose that $G(V, E)$ is a n -vertex graph with a positive length $l(e)$ on each edge e and a positive weight $w(v)$ on each vertex v . For any positive integer p , the well-known p -Center problem is to identify p vertices of G for locating some kind of facilities such that the maximum distance from any vertex without facility to access its nearest facility is minimized. The authors in [23] originally proposed a more practical and important consideration: the induced subgraph of the selected p vertices must be connected and called this new issue as the Connected p -Center problem (the CpC problem). After then, the CpC problem was shown to be NP-Hard on both bipartite graphs and split graphs, and an $O(n)$ -time algorithm on trees, where $w(v) = 1$, for all $v \in V$, was designed. This paper first shows that the CpC problem is also NP-Hard on planar graphs. Then, we show that the CpC problem can be solved in $O(pn)$ -time on trees even $w(v)$ can be any positive number, for all $v \in V$.

Keywords: connected p -centers, graphs with forbidden vertices, induced subgraphs, planar graphs, NP-Hard

1. Introduction

Let $G(V, E, l)$ be an undirected and connected graph with n -vertex-set V and m -edge-set E , where l is a function mapping each edge e to a positive length $l(e)$. Let $d(x, y)$ be the distance of the shortest path between any two vertices x and y . We define $d(x, y) = 0$, if $x = y$. For any $Q \subset V$, $v \notin Q$, define $d(v, Q) = \min_{u \in Q} \{d(v, u)\}$ and $\delta(Q) = \max_{v \in V-Q} \{d(v, Q)\}$. The p -Center problem is defined as follows.

The p -Center problem: [9] Given a graph $G(V, E, l)$ and a positive integer p , identify a subset $H = \{h_1, \dots, h_p\}$ of V , called a p -center of G , such that $\delta(H)$ is minimized.

The p -Center problem has been an essential for wide-area applications to real-world systems and environments. Typical application areas include finding the best locations of various types of facilities such as industrial plants, warehouses, distribution

centers, and public service facilities in transportation networks. Deriving good locations of routers, servers in telecommunication and computer networks is also directed related to the p -Center problem. Due to its theoretical and practical importance, rich research effort has been focused on the p -Center problem and its related problems [1, 2, 3, 5, 7, 8, 10, 15 - 20, 24]. In [23], a survey of important algorithmic results about the p -Center problem has been done. For example, it is NP-Hard on general graphs, $O(n)$ -time solvable on interval graphs, and $O(pn)$ -time solvable on circular-arc graphs, etc. In addition, the authors in [6, 21, 22] dealt with the problem of finding facility vertices on trees that the selected facility vertices are required within a path or a subtree with the length no more than a given constant L .

In [23], the authors stated that in many real-world applications, such as determining the locations of web cash servers, requiring that the p center vertices induce a connected subgraph is important and practical. Indeed, this paper will resolve a more practical and general situation. Let $G(V, E, l, w)$ denote a graph with a positive length $l(e)$ on each edge e and a positive weight $w(v)$ on each vertex v . For any $x, y \in V$, define $d^*(x, y) = w(x) * d(x, y)$ and $d^*(y, x) = w(y) * d(y, x)$. Similarly, for any $Q \subset V$, $v \notin Q$, we define $d^*(v, Q) = \min_{u \in Q} \{d^*(v, u)\}$ and $\beta(Q) = \max_{v \in V-Q} \{d^*(v, Q)\}$. Note that $d^*(x, y) \neq d^*(y, x)$ if $w(x) \neq w(y)$. The problem studied in this paper can now be formally defined as follows:

The Connected p -Center Problem (The CpC problem): Given a graph $G(V, E, l, w)$ and a positive integer p , identify a p -center $Q = \{q_1, \dots, q_p\}$ such that $\beta(Q)$ is minimized under the restriction that the subgraph induced by the vertices in Q must be connected.

In the rest of this paper, given any p -vertex subset $H = \{h_1, \dots, h_p\}$ of V , we just call H as a p -center. If the subgraph induced by H is connected, then H is called a *connected p -center*. Furthermore, if $\beta(H)$ is minimized, then H is called an *optimal connected p -center*, or simply an *optimal solution* of

the graph G .

The CpC problem has been shown to be NP-Hard on both bipartite graphs and split graphs, and an $O(n)$ -time algorithm on trees has been designed in which $w(v) = 1$, for all $v \in V$ [23]. This paper will first show that the CpC problem is also NP-Hard on planar graphs in Section 2. Then, Section 3 will show that the CpC problem can be solved in $O(pn)$ -time on trees such that $w(v)$ can be any positive number, for all $v \in V$. Finally, Section 4 will describe the conclusions and some possible future research issues

2. The CpC Problem on Planar Graphs

In the rest of this paper, for any graph G , $V(G)$ and $E(G)$ denotes the vertex-set and edge-set of G , respectively. In addition, $\deg(v)$ denotes the degree of each vertex v in G and $N(v) = \{u \mid (u, v) \in E(G)\}$. This section will show that the CpC problem is NP-Hard when the input graphs are restricted to planar graphs. A graph $G(V, E)$ is *planar* if G can be drawn into the plane so that the edge-crossings could only occur at vertices [12]. The following is the well-known theorem for testing the planarity of any graph, where K_5 is any clique with 5 vertices and $K_{3,3}$ represents a complete-bipartite graph with vertex-set $X \cup Y$ such that $|X| = |Y| = 3$.

Theorem 1 (Kuratowski's Theorem). [13] A graph G is a planar graph if and only if H is not homeomorphic with K_5 or $K_{3,3}$, for all subgraphs H of G .

To complete our proof, we consider the following decision version of our problem restricted to $w(v) = 1$, for all $v \in V$.

The CpC decision problem: [23] Given a graph $G(V, E)$ in which each edge e is associated with a positive

length $l(e)$, an integer $p \geq 1$, and a constant $\lambda > 0$, the goal is to determine whether a connected p -center Q of G exists such that $\delta(Q) = \max_{v \in V-Q} \{d(v, Q)\} \leq \lambda$.

The following is the NP-complete problem [4, 11, 14] used for reduction.

The Planar Monotone 3SAT problem (The PM3SAT

problem): Let $C = \{c_1, \dots, c_r\}$ denote the set of clauses of a Boolean formula in CNF over Boolean variable-set $X = \{x_1, \dots, x_q\}$. Each clause c_i contains at most three literals, which are all either only positive literals, say x_j , or only negative literals, say $\overline{x_j}$. Meanwhile, the bipartite graph $G^B(V^B, E^B)$, where $V^B = C \cup X$ and $E^B = \{(c_i, x_j) \mid \text{for all } c_i \text{ contains } x_j \text{ or } \overline{x_j}\}$, is also planar. Determine whether there exists an assignment of X satisfying all clauses in C ?

Lemma 1. The CpC decision problem is NP-complete even restricted on planar graphs, for all $p \geq 1$.

Proof: Given $C = \{c_1, \dots, c_r\}$ and $X = \{x_1, \dots, x_q\}$ for the PM3SAT problem, let $C^{(P)} = \{c_1, \dots, c_\alpha\}$ be the set of clauses consisting of positive literals and $C^{(N)} = \{c_{\alpha+1}, \dots, c_r\}$ be the set of clauses consisting of negative literals. It is trivial to solve the PM3SAT problem if one of $C^{(P)}$ and $C^{(N)}$ is empty or $q = 1$. Thus, it is reasonable to assume that both $C^{(P)}$ and $C^{(N)}$ are nonempty

and $q \geq 2$.

Let $\overline{X} = \{\overline{x_1}, \dots, \overline{x_q}\}$, $A = \{a_1, \dots, a_q\}$,

$B = \{b_1, \dots, b_q\}$, and $D = \{d_1, \dots, d_q\}$. For any

constant $\lambda > 0$, the graph $G(V, E, l)$ is constructed by extending from G^B as follows: (1) $V = C \cup X \cup$

$\overline{X} \cup A \cup B \cup D$. (2) $E = \{(c_i, x_j) \mid \text{for all } c_i$

contains $x_j\} \cup \{(c_i, \overline{x_j}) \mid \text{for all } c_i$ contains

$\overline{x_j}\} \cup E_1 \cup E_2$, where $E_1 = \{(x_j, a_j), (a_j,$

$b_j), (b_j, \overline{x_j}) \mid 1 \leq j \leq q\}$ and $E_2 = \{(d_j,$

$x_{j-1}), (d_j, \overline{x_{j-1}}), (d_j, x_j), (d_j, \overline{x_j}) \mid 2 \leq j \leq$

$q\}$ (3) $l(e) = \frac{1}{3}\lambda$, for all edges $e \in E_1$, $l(e) = 2\lambda$,

for all edges $e \in E_2$, and $l(e) = \lambda$, for all other edges e .

Examining the construction rules of G , it is easy to verify that G contains no subgraph homeomorphic with K_5 . According to Kuratowski's Theorem, the remaining task of showing the planarity of G is to ascertain that G contains no subgraph homeomorphic with $K_{3,3}$. Suppose there exists a subgraph H of G

such that H is homeomorphic with $K_{3,3}$ and $V(H)$

$= Y \cup Z$. Since $\deg(a_j) = \deg(b_j) = 2$, $1 \leq j \leq q$, it

implies that $V(H) \cap (A \cup B) = \emptyset$, i.e., $V(H) \subseteq (C \cup D \cup X \cup \overline{X})$. We further claim that $V(H) \cap D = \emptyset$.

Otherwise, there exists $d_j \in V(H)$. Because Y and

Z are symmetric, we can only consider $d_j \in Y$ and

assume that $Y = \{d_j, u, v\}$. Because $N(d_j) = \{x_{j-1},$

$\overline{x_{j-1}}, x_j, \overline{x_j}\}$, $2 \leq j \leq q$, the definition of $K_{3,3}$

and the construction rules of G imply that $u, v \in (C$

$\cup D)$ and $Z \subseteq \{x_{j-1}, \overline{x_{j-1}}, x_j, \overline{x_j}\}$. This also

implies that $|N(u) \cap Z| \geq 3$ and $|N(v) \cap Z| \geq$

3. However, the construction rules of G again implies

that $|N(u) \cap Z| \leq 2$ and $|N(v) \cap Z| \leq 2$. A

contradiction occurs.

The above discussion means that $V(H) \subseteq (C \cup X \cup \overline{X})$. But, it is easy to check that any six

vertices in $C \cup X \cup \overline{X}$ can not induce a $K_{3,3}$. A

contradiction occurs again. We now can make sure that G contains no subgraph homeomorphic with

$K_{3,3}$. Therefore, G is a planar graph and it is clear

that G can be constructed in polynomial-time with respect to r and q .

Next, we will show that there exists a connected p -center Q of G such that $\delta(Q) \leq \lambda$, where $p = 2q - 1$, if and only if the Boolean formula consisting of all clauses in C is satisfiable. Suppose that Q is connected $(2q - 1)$ -center of G such that $\delta(Q) \leq \lambda$.

Since the lengths of all edges incident with each d_j

are equal to 2, it is obvious that $D \subset Q$. Then, we

claim that for each pair x_j and $\overline{x_j}$, at least one of

them must belong to Q . Let there exist j such that

$x_j \notin Q$ and $\overline{x_j} \notin Q$. Because the vertices in Q

induces a connected subgraph, we have $a_j \notin Q$

and $b_j \notin Q$. Let u_j and v_j be the nearest vertices in Q with respect to a_j and b_j , respectively. The construction rules of G imply that $d(u_j, a_j) > \lambda$ and $d(v_j, b_j) > \lambda$. This contradicts to the fact that $\delta(Q) \leq \lambda$. Since $|Q - D| = p$ and Q contains at least one vertex of each pair $\{x_j, \overline{x_j}\}$, $1 \leq j \leq p$, this can imply that Q contains exactly one vertex in each pair $\{x_j, \overline{x_j}\}$, $1 \leq j \leq p$. Besides, both $C^{(P)}$ and $C^{(N)}$ are nonempty, we have $|Q \cap X| \geq 1$ and $|Q \cap \overline{X}| \geq 1$. Combining the fact that $D \subset Q$, Q induces a connected subgraph and each clause c_i has a nearest vertex either x_t or $\overline{x_t}$, for some t . Let the literals corresponding to the vertices in Q be TRUE. This will satisfy the input Boolean formula.

Assume that there is an assignment of X satisfying the input Boolean formula. Without a loss of generality, let $x_1 = \dots = x_k = \text{TRUE}$ and $x_{k+1} = \dots = x_q = \text{FALSE}$. Set $Q = \{x_1, \dots, x_k, \overline{x_{k+1}}, \dots, \overline{x_q}, d_2, \dots, d_q\}$. Because both $C^{(P)}$ and $C^{(N)}$ are nonempty, we have $|Q \cap X| \geq 1$ and $|Q \cap \overline{X}| \geq 1$. By the construction rules of G , it is simple to verify that the subgraph induced by Q must be connected. Furthermore, it is also easy to check that $\delta(Q) \leq \lambda$. This means that Q a connected $(2q - 1)$ -center of G such that $\delta(Q) \leq \lambda$.

Finally, let $G^*(V^*, E^*, l^*)$ be the graph as follows: (1) $V^* = V \cup \{d_1\}$. (2) $E^* = E \cup \{(d_1, x_1), (d_1, \overline{x_1})\}$. (3) $l^*(d_1, x_1) = l^*(d_1, \overline{x_1}) = 2\lambda$ and $l^*(e) = l(e)$, for all other edges e . Using the same reasoning as above, we can show that there exists a connected $(2q)$ -center Q of G such that $\delta(Q) \leq \lambda$ if and only if the Boolean formula consisting of all clauses in C is satisfiable.

All reasoning so far completes our proof. ■

Theorem 1. The CpC problem is NP-Hard even the input graphs are restricted to planar graphs.

3. The CpC Problem on Trees

Let T be a n -vertex tree with a positive length $l(e)$ on each edge e and a positive weight $w(v)$ on each vertex v . The cases $n = 1$ and $n = 2$ are so trivial, we assume that $n \geq 3$. Choose any vertex r as the root and denote T as $T(r)$. To describe our ideas clearly, the following definitions on $T(r)$ are introduced. For each x in $V - \{r\}$, $\text{par}(x)$ denotes the parent vertex of x . For each non-leaf vertex u , $\text{Children}(u)$ denotes the set of child vertices of u and $T(u)$ represents the subtree rooted at u . In addition, $T(u)$ will also represent the vertex-set of $T(u)$ whenever it does not cause any confusion.

To design our algorithm, we do the following pre-processing to obtain some useful information. Given any vertex u of $T(r)$, the first task is able to compute $d(x, u)$, for all $x \in T(u)$. This task can be easily done in $O(|T(u)|)$ -time according to the following recurrence formulas.

$$d(u, u) = 0.$$

$$d(x, u) = d(\text{par}(x), u) + l(x, \text{par}(x)), \text{ for all } x \in$$

$T(u)$ and $x \neq u$.

For each vertex u , let $f(u)$ be a vertex in $T(u)$ such that $d^*(f(u), u) \geq d^*(y, u)$, for all $y \in T(u)$. The following code segment is designed to identify $f(u)$ for any given vertex u . The approach is based upon the *level order traversal* on each vertex of $T(u)$ from the vertex u . Its correctness can be easily verified from the properties of trees and the time-complexity is $O(|T(u)|)$.

```

if ( $u$  is a leaf vertex) then  $\text{return}((u, 0))$ ;
//  $f(u)$  is just the vertex  $u$  itself and  $d^*(f(u), u) = 0$ .
   $\text{Compute\_Distances}(u)$ ;
  //  $\text{Compute\_Distances}(u)$  computes  $d(x, u)$ , for
  all  $x \in T(u)$ .
   $H = \text{Children}(u)$ ;
   $\text{id} = \text{NULL}$ ;  $\text{dist} = 0$ ;
  while ( $H$  is not empty)
     $Q = \emptyset$ ;
    for each  $x \in H$  do
      if ( $w(x) * d(x, u) > \text{dist}$ )
         $\text{id} = x$ ;  $\text{dist} = w(x) * d(x, u)$ ;
      endif
      if ( $x$  is not leaf)
         $Q = Q \cup \text{Children}(x)$ ;
      endif
    endfor
     $H = Q$ ;
  endwhile
   $\text{return}((\text{id}, \text{dist}))$ ;
//  $\text{id}$  is the vertex  $f(u)$  and  $\text{dist}$  is equal to
 $d^*(f(u), u)$ .

```

It is the time for dealing with the following restricted problem.

Problem $P^{(r)}$: Given a tree $T(r)$, find an optimal connected p -center Q under the constraint that $r \in Q$.

We design the following algorithm for solving the problem $P^{(r)}$.

Algorithm_ $P^{(r)}$

Input: (1) A n -vertex tree $T(r)$ with a positive length $l(e)$ on each edge e and a positive weight $w(v)$ on each vertex v . (2) A positive integer $p \leq n$.

Output: A connected p -center Q such that $\beta(Q)$ is minimized under the constraint that $r \in Q$.

begin

```

   $Q = \{r\}$ ;
   $j = 1$ ;
   $\Omega = \{T(r)\}$ ;
  while ( $j < p$ )
    find  $T(y)$  such that  $d^*(f(y), y)$  is
    maximized among all subtrees in  $\Omega$ ;
    let  $x$  be the vertex in  $\text{Children}(y)$  such
    that  $f(y) \in T(x)$ ;
     $Q = Q \cup \{x\}$ ;
     $\Omega = \Omega - \{T(y)\}$ ;
     $\Omega = \Omega \cup \{T(x)\} \cup \{T(y) - T(x)\}$ ;
     $j++$ ;
  endwhile
   $\text{return}(Q)$ ;
  //  $\beta(Q)$  is equal to  $d^*(f(x), x)$ , where  $x$  is the
  last vertex added to  $Q$ .

```

end

End_Algorithm_ $P^{(r)}$

Theorem 2: Algorithm_ $P^{(r)}$ can solve the problem $P^{(r)}$ correctly and its time-complexity is $O(pn)$.

Proof: It is easy to see that the time-complexity of each iteration of the loop, **while** ... **endwhile**, is

$$\sum_{T(z) \in \Omega} O(T(z)) = O(n). \text{ The body of the loop}$$

will be repeated $(p - 1)$ times. This implies that the total time-complexity of $\text{Algorithm_P}^{(r)}$ is $O(pn)$.

We now show the correctness of $\text{Algorithm_P}^{(r)}$. Let x_j , $1 \leq j \leq (p - 1)$, be the vertices added to Q during the j^{th} iteration of the **while ... endwhile** loop, y_j is the parent of x_j , and Q_j be the set obtained after the j^{th} iteration. Initially, $Q_0 = \{r\}$. It is easy to verify that each Q_j induces a connected subgraph and $\beta(Q_{j-1} \cup \{x_j\}) \leq \beta(D)$, for all connected $(j + 1)$ -centers D of $T(r)$ with $r \in D$, for all $1 \leq j \leq (p - 1)$. This implies that the output $Q = Q_{p-1}$ is certainly an optimal connected p -center of $T(r)$. ■

The left problem is finding a vertex r of T such that r must be included into some optimal solution. To achieve the goal, the following lemma is established.

Lemma 2: Assume that u and v are two vertices satisfying $d^*(u, v) \geq d^*(x, y)$, for all $x, y \in V$. Let m be the point of the path from u to v such that $w(u) * d(u, m) = w(v) * [d(u, v) - d(u, m)]$, i.e., $d(u, m) = \frac{w(v) * d(u, v)}{w(u) + w(v)}$. Let r be a vertex nearest to m .

Suppose that Q is a connected p -center of the tree such that $r \in Q$. Then, $\beta(Q) \leq \beta(H)$, for all connected p -center H with $r \notin H$.

Proof: The definition of the vertex r implies that m must lie on some edge incident with r . Pick r as the root of the input tree and assume that $\text{Children}(r) = \{x_1, \dots, x_k\}$. We just handle the case that m is an

internal point of some edge (r, x_j) , $1 \leq j \leq k$, i.e., $m \neq r$ and $m \neq x_j$. Verifying that $\{r, x_j\} \subseteq Q$ and

$$\beta(Q) \leq \max\{d^*(v, x_j), d^*(u, r)\} < w(v) * d(v, m)$$

is simple. Consider any connected p -center H with $r \notin H$. The definitions of trees and connected p -center imply that $H \subseteq V(T(x_j))$, for some subtree $T(x_j)$. Without a loss of generality, we assume that $v \in T(x_j)$ and $u \in T(x_{j+1})$. Then, one of the following cases could occur.

Case 1: $H \subseteq V(T(x_j))$

It is easy to verify that $\beta(H) \geq w(u) * (d(u, m) + d(m, x_j)) = w(u) * d(u, m) + w(u) * d(m, x_j) = w(v) * d(v, m) + w(u) * d(m, x_j) > \beta(Q)$.

Case 2: $H \subseteq V(T(x_s))$, for some s , $1 \leq s \neq j \leq k$

In this case, $\beta(H) \geq w(v) * d(v, r) + w(v) * d(r, x_s) > w(v) * d(v, m) + w(v) * d(r, x_s) > \beta(Q)$.

The above reasoning can ascertain that $\beta(H) \geq \beta(Q)$. ■

The final task is to derive u and v such that $d^*(u, v) \geq d^*(x, y)$, for all $x, y \in V$. After that, let r be the vertex as described in Lemma 2. Then, we can just use $\text{Algorithm_P}^{(r)}$ to identify an optimal connected p -center of the input tree T .

Now, choose any vertex u as the root of T . For each vertex y of $T(u)$, three values $\mu(y)$, $\Delta^{(m)}(y)$, and $\Delta^{(s)}(y)$ are computed by the following rules.

- (1) $\mu(y) = l(y, \text{par}(y))$, $\Delta^{(m)}(y) = 0$, and $\Delta^{(s)}(y) = 0$, if y is not the root u and y is a leaf vertex.
- (2) $\mu(y) = \max_{x \in \text{Children}(y)} \{\mu(x)\} + l(y, \text{par}(y))$,
 $\Delta^{(m)}(y) = \max_{x \in \text{Children}(y)} \{\mu(x)\}$, and
 $\Delta^{(s)}(y) = \sec_{x \in \text{Children}(y)} \{\mu(x)\}$, where

$\text{sec}_{x \in \text{Children}(y)} \{\mu(x)\}$ returns the second largest value among $\mu(x)$, for all $x \in \text{Children}(y)$, if y is a non-leaf vertex other than the root u .

(3) $\mu(y) = \max_{x \in \text{Children}(y)} \{\mu(x)\}$, $\Delta^{(m)}(y) = \mu(y)$, and $\Delta^{(s)}(y) = \text{sec}_{x \in \text{Children}(y)} \{\mu(x)\}$, if y is the root u .

Examining the above rules carefully, for each non-leaf vertex y of $T(u)$, $\Delta^{(m)}(y) = d(y, z)$, for some $z \in T(y)$, such that $d(y, z) \geq d(y, x)$, for all $x \in T(y)$, i.e., $\Delta^{(m)}(y)$ is the distance of the path from y to its farthest vertex z within $T(y)$. Using the *depth-first-search* on the tree $T(u)$ from the root u , the following lemma can be easily proved.

Lemma 3: The values, $\mu(y)$, $\Delta^{(m)}(y)$, and $\Delta^{(s)}(y)$, for all vertices y in $T(u)$ can be obtained in $O(n)$ -time totally.

Next, for each vertex y of $T(u)$, another value $\theta(y)$ is computed as follows:

- (1) $\theta(y) = 0$, if y is the root u .
- (2) $\theta(y) = \max\{\theta(\text{par}(y)), \Delta^{(s)}(\text{par}(y))\} + l(y, \text{par}(y))$, If y is not the root u .

It is easy to check that for each non-root vertex y of $T(u)$, $\theta(y) = d(y, z)$, for some $z \in (T(u) - T(y))$, such that $d(y, z) \geq d(y, x)$, for all $x \in (T(u) - T(y))$, i.e., $\theta(y)$ is the distance of the path from y to its farthest vertex z within $(T(u) - T(y))$. Also, by the *depth-first-search* on the tree $T(u)$ from the root u , the following lemma can be easily proved.

Lemma 4: The values $\theta(y)$, for all vertices y in $T(u)$, can be obtained in $O(n)$ -time totally.

Now, the two vertices u and v such that $d^*(u, v) \geq d^*(x, y)$, for all $x, y \in V$, can be easily obtained by the following code segment.

```

id = NULL; dist = 0;
for each vertex  $y \in V$ 
    if ( $w(y) * \max\{\theta(y), \Delta^{(m)}(y)\} > \text{dist}$ )
        id =  $y$ ; dist =  $w(y) * \max\{\theta(y), \Delta^{(m)}(y)\}$ ;
    endif
endfor
u = id;
if dist =  $w(y) * \theta(y)$ 
    v is the vertex corresponding to  $\theta(y)$ ;
else
    v is the vertex corresponding to  $\Delta^{(m)}(y)$ ;

```

```

endif
return ((u, v));

```

The following lemma holds consequently.

Lemma 5: Give a tree T , the two vertices u and v such that $d^*(u, v) \geq d^*(x, y)$, for all $x, y \in V$, can be found in $O(n)$ -time.

Based upon all discussion so far, we can design an algorithm to solve the CpC problem on trees correctly as follows:

Algorithm Connected_p_Center_on_Trees

Input: (1) A n -vertex tree T with a positive length $l(e)$ on each edge e and a positive weight $w(v)$ on each vertex v . (2) A positive integer $p \leq n$.

Output: A connected p -center Q such that $\beta(Q)$ is minimized.

begin:

find a pair of vertices u and v such that

$d^*(u, v) \geq d^*(x, y)$, for all $x, y \in V$;

identify the root r such that r is a vertex nearest to the point m of the path from u to v as described in Lemma 2;

$Q = \text{Algorithm_P}^{(r)}(T, p, r)$;

return Q ;

end

End Connected_p_Center_on_Trees

All reasoning derived so far can establish the following main theorem.

Theorem 3: The CpC problem on n -vertex trees with positive lengths on edges and positive weights on vertices can be solved in $O(pn)$ time.

4. Conclusions

The CpC problem was originally proposed and studied in [23]. This problem has been known to be NP-Hard on bipartite graphs and split graphs, and an $O(n)$ -time algorithm on trees exists for the situation that $w(v) = 1$, for all $v \in V$. This paper showed that the CpC problem is also NP-Hard on planar graphs. Then, we showed that the CpC problem can be

solved in $O(pn)$ -time on trees even $w(v)$ can be any positive number, for all $v \in V$.

In the future, the first practical and meaningful issue is to extend our algorithms to cactus graphs and block graphs. Meanwhile, solving the problem on other classes of graphs, such as interval graphs and permutation graphs, is also a very typical research topic. Identifying other variants of the traditional p -Center problem is also a very practical and interesting task. For example, requiring that subgraph induced by the p -centers has no isolated vertices, is another typical issue with significant applications to computer networks.

References

- [1] Abdelaziz F. (2006), 1-center problem on the plane with uniformly distributed demand points, *Operations Research Letters*, Vol. 34, Iss. 3, 264-268.
- [2] Averbakh I. and Berman O. (1995), Sales-delivery man problems on treelike networks, *Networks*, Vol. 25, 45-58.
- [3] Bar-Ilan J, Peleg D (1991), Approximation algorithms for selecting network centers. in *Proceedings of workshop on algorithms and data structures*, 343–354.
- [4] Barrett C., Hunt III H. B., Marathe M. V., Ravi S. S., Rosenkrantz D. J., Stearns R. E., and Thakur M. (2007), Predecessor existence problems for finite discrete dynamical systems, *Theoretical Computer Science*, Vol. 386, 3-37.
- [5] Bspamyatnikh S, Bhattacharya B, Keil M, Kirkpatrick D, Segal M. (2002), Efficient algorithms for centers and medians in interval and circular-arc graphs, *Networks*, Vol. 39, 144–152.
- [6] Bhattacharya B. K., Hu Y., Shi Q., and Tamir A. (2006), Optimal algorithms for the path/tree-shaped facility location problems in trees. in *Proceedings of International Symposium on Algorithm and Computation*, 379–388.
- [7] Burkard R. E. and Dollani Helidon (2003), Center problems with pos/neg weights on trees, *European Journal of Operational Research*, Vol. 145, Iss. 3, 483-495
- [8] Chenga T. C. E., Kang L., and Ng C. T. (to appear), An improved algorithm for the p -center problem on interval graphs with unit lengths, *Computers & Operation Research*.
- [9] Daskin M. S. (1995), *Networks and Discrete Location, Models, Algorithms, and Applications*, John Wiley & Sons, Inc., New York.
- [10] Frederickson G. (1991), Parametric search and locating supply centers in trees, in *Proceedings of workshop on algorithms and data structures*, 299–319.
- [11] Garey M. R. and Johnson D. S. (1978), *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Bell Laboratories, Murray Hill, Freeman & Co., N. J.
- [12] Golumbic M. C. (1980), *Algorithmic Graph Theory and Perfect Graphs*, Academic Press, Inc., New York.
- [13] Gould R., (1988), *Graph Theory*, The Benjamin/Cummings Publishing Company, Inc., Menlo Park, California.
- [14] Hunt III H. B., Marathe M. V., Radhakrishnan, and Stearns R. E. (1998), The complexity of planar counting problems, *SIAM Journal on Computing*, Vol. 27, No. 4, 1142-1167.
- [15] Hsu V. N., Lowe T. J., Tamir A. (1997), Structured p -facility location problems on the line solvable in polynomial time, *Operations Research Letters*, Vol. 21, 159–164.
- [16] Kariv O, Hakimi S. L. (1979), An algorithmic approach to network location problems I: the p -centers, *SIAM Journal of Applied Mathematics*, Vol. 37, 514–538.
- [17] Lan Y-F, Wang Y-L (2000), An optimal algorithm for solving the 1-median problem on weighted 4-cactus graphs, *European Journal of Operational Research*, Vol. 122, 602-610.
- [18] Lan Y-F, Wang Y-L, Suzuki H. (1999), A linear-time algorithm for solving the center problem on weighted cactus graphs, *Information Processing Letters*, Vol. 71, 205-212.
- [19] Olariu S. (1990), A simple linear-time algorithm for computing the center of an interval graph, *International Journal of Computer Mathematics*, Vol. 24, 121–128.
- [20] Tamir A. (1988), Improved complexity bounds for center location problems on networks by using dynamic data structures, *SIAM Journal of Discrete Mathematics*, Vol. 1, 377–396.
- [21] Tamir A., Puerto J., and Pérez-Brito D. (2002), The centroid subtree on tree networks, *Discrete Applied Mathematics*, Vol. 118, 263-278.
- [22] Tamir A., Puerto J., Mesa J. A., and Rodríguez-Chía. A. M. (2005), Conditional location of path and tree shaped facilities on trees, *Journal of Algorithms*, Vol. 56, 50-75.
- [23] Yen W. C-K and C-T Chen (2007), The p -Center Problem with Connectivity Constraint, *Applied Mathematical Sciences*, Vol. 1, No. 27, 1311-1324.
- [24] Zmazek B. and Žerovnik J. (2004), The obnoxious center problem on weighted cactus graphs, *Discrete Applied Mathematics*, Vol. 136, 377-386.